Framework Mapping and Build Plan

Role: Researcher + Technical Head

Objective: build a startup research and evaluation system that helps entrepreneurs understand whether their venture has the qualities of the top 10 percent of successful startups.

The system will use Gemini as a background research engine, but the actual intelligence of the product will come from our framework, scoring logic, evidence grading, and agent workflow.

1. What We Are Making

We are making a startup evaluation agent.

The user gives a startup idea. The system researches it, maps it to our framework, scores it, explains what is missing, and gives the founder a practical validation plan.

The product should answer:

- Is this startup idea worth pursuing?
- Why this?
- Why now?
- Why this team?
- Why might this fail?
- Who is the customer?
- How does the startup create value?
- How does it capture value?
- What evidence exists?
- What evidence is missing?
- What should the founder do next?

2. The Main Framework

Framework name:

```
10 Percent Startup Readiness Framework
```

Core idea:

```
A startup becomes a strong candidate when it creates real customer value,
captures that value sustainably, has the right timing, has the right team,
and has evidence stronger than founder belief.
```

3. Framework Map

Layer 1: Venture Foundation

This checks whether the idea is understandable and specific.

Dimensions:

- Problem

- Customer
- Solution
- Value proposition
- Business model
- Stage

Key output:

```
We help X do Y by doing Z.
```

Example:

```
We help independent restaurants reduce food waste by predicting daily demand using sales, weather, and
local event data.
```

Layer 2: Value Loop

This checks whether the startup can create and capture value.

Questions:

- What value is created?
- Who receives the value?
- Is the value urgent or measurable?
- How does the startup capture value?
- Can the value loop repeat?

Scoring:

- 1 = unclear value
- 3 = plausible value but not proven
- 5 = measurable value with clear willingness to pay

Layer 3: The 3+1 Why Test

This is the strategic logic layer.

Questions:

- Why this?
- Why now?
- Why this team?
- Why won't this work?

This prevents shallow startup analysis. A good idea is not enough. The system must understand timing, team fit, and failure modes.

Layer 4: Four-Factor Success Fit

This is the MIT-style success layer.

Dimensions:

- Ideas
- Execution
- Timing
- People

Rule:

```
All four must align. A startup can fail if even one is weak.
```

Examples:

- Strong idea + weak execution = slow death
- Strong team + wrong timing = too early
- Strong product + unclear people fit = conflict or poor market learning

Layer 5: Customer Truth

This checks whether the founder knows the real customer.

Customer map:

- User
- Buyer
- Decision-maker
- Influencer
- Blocker
- Existing alternative

The system must punish vague customer definitions.

Bad:

```
Our customer is students.
```

Better:

```
The daily user is engineering students. The buyer may be the student, parent, university department,
or test-prep platform. This is not yet validated.
```

Layer 6: Evidence Ladder

This is the most important layer for serious evaluation.

Evidence levels:

- Level 0: founder opinion
- Level 1: secondary research
- Level 2: customer interviews
- Level 3: prototype usage
- Level 4: paid pilot
- Level 5: repeatable revenue or retention

Rule:

```
The score cannot be high if the evidence level is low.
```

Layer 7: Failure Anticipation

This checks what can kill the startup.

Risk categories:

- Market risk
- Customer risk
- Timing risk
- Team risk
- Execution risk
- Business model risk
- Legal/regulatory risk
- Capital risk
- Defensibility risk

The system should always ask:

```
Why won't this work?
```

4. Final Scorecard

Use a 1 to 5 score for each dimension.

Dimension	Weight	What It Measures
Problem Pain	1.2	Is the problem urgent, costly, frequent, or emotionally painful?
Customer Clarity	1.2	Do we know the user, buyer, decision-maker, and blocker?
Value Creation	1.1	Does the solution create measurable value?
Value Capture	1.1	Can the startup capture money or strategic value?
Why This	1.0	Is this idea worth pursuing?
Why Now	1.0	Is the timing right?
Why This Team	1.2	Is this team unusually suited to win?
Execution Readiness	1.1	Can the team build, sell, and learn quickly?
Timing	1.0	Is the market ready?
Defensibility	0.9	Is the startup hard to copy or replace?
Risk Awareness	1.0	Does the founder understand what can kill the startup?

Dimension	Weight	What It Measures
Evidence Quality	1.4	Is there real evidence beyond opinions?

Score interpretation:

- 4.2 to 5.0 = strong 10 percent candidate
- 3.5 to 4.1 = promising but has clear gaps
- 2.8 to 3.4 = interesting but under-validated
- 2.0 to 2.7 = weak readiness
- 1.0 to 1.9 = not ready as a venture

5. Agent Map

Agent 1: Intake Agent

Job:

Collect and structure the founder's idea.

Inputs:

- Idea
- Problem
- Customer
- Solution
- Business model
- Team
- Stage
- Geography
- Industry

Output:

```
{
"startup_name": "",
"problem": "",
"solution": "",
"target_user": "",
"target_buyer": "",
"business_model": "",
"team_strength": "",
"stage": "",
"unknowns": []
}
```

Agent 2: Framework Mapper

Job:

Convert the startup profile into framework questions.

Example:

```
{
"dimension": "Customer Clarity",
"questions": [
"Who is the daily user?",
"Who owns the budget?",
"Who can block adoption?",
"What alternative is used today?"
]
}
```

Agent 3: Gemini Research Agent

Job:

Use Gemini in the background to research external facts.

Research should be limited to the framework questions.

It should research:

- Market trends
- Competitors
- Substitutes
- Pricing benchmarks
- Regulation
- Adoption timing
- Industry pain points

It should not invent:

- Customer willingness to pay
- Founder ability
- Product performance
- Pilot results

Those must be marked as validation gaps.

Agent 4: Evidence Normalizer

Job:

Turn research into clean evidence.

Every evidence item should have:

- Claim
- Source
- Framework dimension
- Confidence
- Whether it is direct or indirect evidence

Agent 5: Scoring Agent

Job:

Score each framework dimension from 1 to 5.

Rules:

- No high score without evidence
- Penalize vague customers
- Penalize unclear business model
- Penalize missing buyer
- Penalize weak differentiation
- Penalize no validation

Agent 6: Gap Analysis Agent

Job:

Tell the founder what the startup lacks.

For every weak area, output:

- What is missing
- Why it matters
- Justification
- What test should be done next

Example:

```
Gap: Customer buyer is unclear.
Why it matters: Users may love the product, but if no one owns the budget, the startup cannot capture value.
Justification: The idea identifies students as users but does not prove whether students, parents, or universities will pay.
Next test: Interview 10 students, 10 parents, and 5 university program heads. Test paid intent with a landing page.
```

Agent 7: Recommendation Agent

Job:

Give the final verdict and action plan.

Outputs:

- Overall score
- Readiness category
- Top 3 gaps
- 7-day validation plan
- 30-day validation plan
- What would increase the score
- Whether to proceed, pivot, or validate more

6. Technical Architecture

Frontend

User screens:

1. Startup idea intake 2. Framework mapping preview 3. Research progress 4. Scorecard report 5. Gap analysis 6. Validation plan 7. Saved startup reports

Backend

Core services:

- User/session service
- Venture profile service
- Agent orchestration service
- Gemini research service
- Evidence store
- Scoring engine
- Report generator

Database Tables

```
users
ventures
venture_profiles
framework_dimensions
research_tasks
research_evidence
scorecards
gap_analysis
recommendations
reports
```

Gemini API Layer

Environment variable:

```
GEMINI_API_KEY
```

Model config:

```
{
"primary_model": "gemini-3.5-flash-lite",
"fallback_model": "gemini-3.8-flash",
"temperature": 0.2,
"response_format": "json"
}
```

The model must return structured JSON.

7. Report Format

Final user report:

```
1. Venture summary
2. Value proposition
3. 3+1 Why analysis
4. Scorecard
5. Overall score
```

```
6. What it lacks
7. Justification for each weak score
8. Evidence found
9. Evidence missing
10. Recommended validation experiments
11. Final verdict
```

8. Example Output

Startup:

```
AI tutor for engineering students
```

Value proposition:

```
We help engineering students prepare for difficult exams by generating personalized practice problems
and explanations.
```

Score:

```
3.6 / 5
Promising but not yet a strong 10 percent candidate.
```

What it lacks:

- Clear economic buyer
- Proof of willingness to pay
- Strong defensibility
- Evidence of improved learning outcomes

Justification:

```
The user is clear, but the buyer is not. Students may use the product, but it is not yet proven
whether students, parents, universities, or tutoring companies will pay. The market timing is strong
because AI adoption is rising, but competition is also high. The idea needs direct validation through
paid pilots and retention data.
```

Next experiments:

- Interview 20 engineering students
- Interview 5 professors or department administrators
- Launch a landing page with pricing
- Run a 2-week exam prep pilot
- Measure usage, completion, willingness to pay, and grade improvement

9. How I Would Build It

Phase 1: Framework Engine

Build the framework in code first.

Deliverables:

- Framework dimensions
- Scoring weights

- Evidence levels
- Score interpretation
- Gap templates

Goal:

Make the system work even before Gemini is connected.

Phase 2: Intake and Profile Builder

Build the founder intake flow.

Deliverables:

- Startup profile form
- Structured venture profile JSON
- Missing information detector

Goal:

Turn messy founder input into clean analysis input.

Phase 3: Gemini Research Agent

Connect Gemini API.

Deliverables:

- Research prompt templates
- JSON response schema
- Citation storage
- Research task queue
- Retry and fallback model handling

Goal:

Make Gemini research only what the framework asks for.

Phase 4: Scoring and Gap Engine

Build scoring logic.

Deliverables:

- Weighted scoring calculator
- Evidence quality adjustment
- Gap detection
- Justification generator

Goal:

Produce trustworthy scores and explain them clearly.

Phase 5: Final Report Generator

Build the report.

Deliverables:

- Scorecard
- Framework analysis
- Missing evidence
- Validation plan
- Final verdict

Goal:

Give founders an output they can act on immediately.

Phase 6: Agent Improvement Loop

Improve with real startup cases.

Deliverables:

- Feedback collection
- Score calibration
- Better industry-specific prompts
- Benchmark examples

Goal:

Make the system smarter over time.

10. My Technical Decision

As technical head, I would not let Gemini control the whole system.

Gemini should be used for:

- Research
- Summarization
- Evidence extraction
- Explanation

Our backend should control:

- Framework
- Scoring weights
- Evidence levels
- Final score
- Gap rules
- Report structure

This makes the system consistent, testable, and defensible.

11. Minimum Viable Product

The first version should do only one thing very well:

```
Take a startup idea and produce a framework-based scorecard with gaps and next validation steps.
```

MVP features:

- Startup idea intake
- Framework scoring
- Gemini research
- Gap analysis
- Final report

Do not start with:

- Full investor matching
- Pitch deck creation
- Financial modeling
- Complex dashboards
- Social features

Those can come later.

12. Success Criteria

The framework works if founders say:

- "I now understand what is weak in my idea."
- "I know what evidence I need next."
- "The score feels justified."
- "The system did not just motivate me; it challenged me."
- "I can use this to decide whether to continue, pivot, or stop."